

ENGINEERING CEREBRO

Serial hardware, RGB-D perception, MLX/VLM, Gemini, and dependable robot shows



AN IMPLEMENTATION-LEVEL GUIDE FOR SWIFT AND OBJECTIVE-C DEVELOPERS

Rodolfo Aramayo / Orbitus Robotics - First Maker Faire Edition

Build a robot mind from bounded, inspectable parts

Cerebro becomes reliable when every camera frame, serial byte, model turn, and actuator request crosses an explicit contract.

This volume reads the current Cerebro repository as a living mixed-language robot system. It dissects ROBSerialBox and its POSIX/IOKit device paths, AVFoundation capture, the isolated Luxonis DepthAI helper, Vision and SceneKit rendering, MLX language and vision-language inference, bounded semantic memory, Gemini Live media and tools, SceneSnapshot, historical Kinect artifacts, safe light-saber choreography, comedy-show state machines, and the separate Vision Pro control/video contract taught in Volume 7.

Paths are relative to the public <https://github.com/RudyAramayo/Cerebro> repository, with the spatial client in <https://github.com/RudyAramayo/ROBControllerVision>. Every chapter names the exact implementation or test being analyzed. Historical artifacts are not automatically recommended, model output is never motor authority, and physical behavior must be verified under a controlled commissioning procedure.

Copyright © 2026 Rodolfo Aramayo / Orbitus Robotics. Photographs are from the private ROB build archive unless otherwise noted. The generated frontispiece is original artwork derived from a ROB reference photograph. This independent educational project is not affiliated with or endorsed by any film studio or franchise owner. Product and company names remain the property of their respective owners.

First evidence-based draft: 2 August 2026. Build details marked **Builder input needed** are deliberate placeholders for the builder to complete.

SOFTWARE CONFIDENCE IS NOT PHYSICAL CERTIFICATION

Cerebro controls high-energy mobile and articulated hardware. A successful build, camera detection, model response, serial banner, green UI state, or passing fixture does not certify wiring, brakes, limits, calibration, collision avoidance, force, stability, or emergency stopping. Test unpowered first, isolate actuators, use conservative limits, maintain a physical emergency stop, and keep one responsible human in command.

The Building R.O.B. library

VOLUME 1	<i>Meet ROB: A Robot Is a Team of Systems</i> - ages 8-12
VOLUME 2	<i>Circuits and Signals with ROB</i> - ages 10-14
VOLUME 3	<i>Motion Workshop: Treads, Gears, and Fabrication</i> - ages 10-15
VOLUME 4	<i>Mission Control: Arduino, Mac, Networks, and Vision</i> - ages 12-16
VOLUME 5	<i>AI, Robotics, and Codex: Directing, Verifying, and Owning AI-Built Systems</i> - advanced makers
VOLUME 6	<i>Dual-Arm Robotics: AMBER B1, URDF, CAN, and Ubuntu</i> - advanced makers
VOLUME 7	<i>Engineering ROBControllerVision: Swift, Spatial Input, Video, and Speech</i> - Swift developers
VOLUME 8	<i>Engineering Cerebro: Perception, Models, H.264, and Robot Coordination</i> - software engineers
MANUAL	<i>Building R.O.B.: The Complete Maker's Field Manual</i> - advanced builders

HOW TO USE THIS BOOK

Volume 8 is the robot-side companion to Volume 7. Read Volume 7 for ROBControllerVision's Swift 6 state model, spatial UI, controller/dead-man input, head pose, paired QUIC control, H.264 reception, and speech. Read this volume for Cerebro's matching server, camera, hardware, model, and performance architecture.

HOW TO FIND THE FILES

Open the public repositories on GitHub and follow each printed repository-relative path: <https://github.com/RudyAramayo/ROBBooks>, <https://github.com/RudyAramayo/ROBArduino>, <https://github.com/RudyAramayo/Cerebro>, <https://github.com/RudyAramayo/ROBController>, <https://github.com/RudyAramayo/ROBControllerVision>, <https://github.com/RudyAramayo/Amber-HomeFolder>, and <https://github.com/RudyAramayo/ROBTrainingGames>. The website is published separately at <https://github.com/OrbitusRoboticsWebSite/0Robotics>. See <https://github.com/RudyAramayo/ROBBooks/blob/main/ROB-Books/OPEN-SOURCE-CODE-MAP.md> for clickable repository and file links, license cautions, and renamed-file search instructions. A named archival item without a verified public repository is labeled as evidence, not given an invented URL.

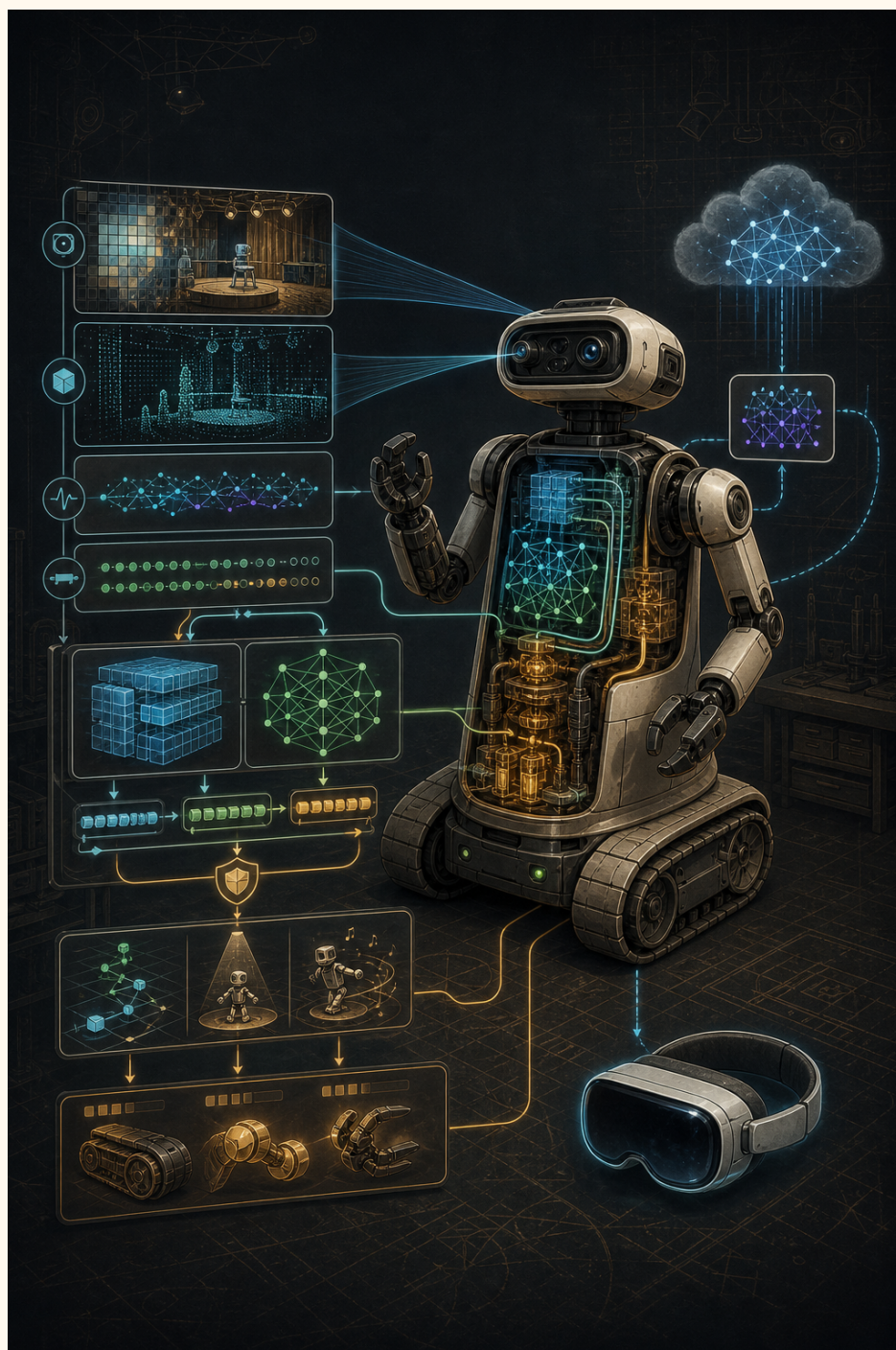


Figure 1: Conceptual illustration: Cerebro admits camera, depth, serial, local-model, cloud-model, show, and Vision Pro data through separate bounded paths. It is a teaching diagram, not a wiring plan or literal screenshot.

Contents

1 Read Cerebro as a living robot system	1
2 Map the mixed-language architecture	2
3 Reproduce the build before changing behavior	3
4 Enter through AppDelegate, then find ownership	4
5 Dissect ROBSerialBox: the hardware boundary	5
5.1 File descriptors are state machines	5
6 Discover the Base Arduino by firmware response	6
6.1 Parse lines before interpreting meaning	6
7 Separate devices by identity and protocol	7
8 Translate controller intent without extending stale motion	8
9 Control Maestro neck servos and Tic torso rotation	9
10 Bridge Objective-C and Swift deliberately	10
11 Capture RGB with AVFoundation	11
12 Acquire aligned RGB-D from Luxonis	12
12.1 Separate host inference from OAK inference	12
12.2 Understand alignment	12
13 Build a bounded perception pipeline	14
14 Render perception without leaking memory	15
15 Turn observations into SceneSnapshot	16
16 Run MLX privately on Apple silicon	17
16.1 MLX VLM—sometimes abbreviated incorrectly as VLX	17
17 Read the MLX execution path line by line	18
18 Teach ROB to recognize a new object without teaching it to trust itself	19
18.1 A safe future learning architecture	20
19 Add bounded semantic memory	21

20 Constrain local model output	22
21 Integrate Gemini Live without coupling safety	23
22 Treat model tools as proposals	24
23 Stream camera video to Vision Pro separately	25
24 H.264 without hand-waving: pictures, access units, and NAL units	26
24.1 Annex B and AVCC are two wrappers around NAL units	26
24.2 Read the encoder from input pixel to encoded bytes	27
24.3 ROB's transport is custom reliable QUIC, not RTP	27
24.4 Why a missing access unit triggers IDR recovery	28
25 Excavate the Kinect and OpenNI past	29
26 Compare Kinect-era and Luxonis-era design	30
27 Build a reliable light-saber battle trainer	31
28 Engineer the comedy show as a state machine	32
29 Test at contracts, not only windows	33
30 Profile memory, latency, and thermals	34
31 Refactor without erasing history	35
32 File-by-file reading route	36
33 Production review checklist	37
34 Closing principle	38

Read Cerebro as a living robot system

Cerebro is the macOS application at the center of ROB. It is not one algorithm. It is a boundary between cameras, serial devices, robot controllers, speech, local models, cloud models, two AMBER arms, a stage-show engine, and a human operator. The public source is <https://github.com/RudyAramayo/Cerebro>. Clone it, then use this repository root:

```
1 Cerebro/
```

Keep Xcode and the repository open while reading. Paths beginning `Cerebro/` are relative to that repository. Tests are under `Tests/`; operational notes are under `docs/`. The companion spatial client is in the sibling `ROBControllerVision/` repository and is taught in Volume 7.

SOURCE TRAIL – ANALYZING NOW: `README.md`, `Cerebro.xcodeproj/project.pbxproj`, `Cerebro/AppDelegate.m`, and `Cerebro/ROBMainViewController.mm` establish the application boundary and composition graph.

This book uses **observed** for behavior supported by the inspected source, **historical artifact** for retained code that is not the preferred current path, and **proposed** for an improvement. Never infer that a compiled control button proves a physical mechanism is safe or calibrated.

Map the mixed-language architecture

Cerebro grew across generations of Apple and robotics technology. Its composition therefore crosses Objective-C, Objective-C++, C++, Swift, Python, shell helpers, JSON, Core ML models, and SceneKit assets.

- **Application shell:** AppDelegate.m and ROBMainWindowController.m own lifecycle, windows, service startup, and dependency checks.
- **Operator coordination:** ROBMainViewController.mm joins camera, speech, transport, shows, and robot state.
- **Physical I/O:** ROBSerialBox.h/.m reaches Base, Maestro, Tic, AMBER, and historical serial roles.
- **Camera and perception:** CameraManager.swift and CameraViewController.swift separate provider lifecycle from Vision/SceneKit presentation.
- **Local and cloud intelligence:** ROBMLXRuntime.swift, ROBAI.swift, and GeminiRoboticsProtocol.swift keep provider concerns separate.
- **Typed world state:** ROBSceneSnapshot.swift describes people, objects, free space, poses, and confidence.
- **Performance:** the ROBStageShow files and ROBSaberChoreography.swift validate cues and gesture names.
- **Spatial remote:** the sibling ROBControllerVision/ repository owns Vision Pro input, video decoding, and operator UI.

Objective-C owns much of the hardware-facing application because it grew from Cocoa-era controller code. Swift supplies newer protocol, concurrency, machine-learning, and media boundaries. The bridging header is therefore architectural infrastructure, not a temporary embarrassment. Keep exposed interfaces narrow, nullability explicit, and callbacks asynchronous.

Reproduce the build before changing behavior

SOURCE TRAIL – ANALYZING NOW: Package.resolved inside the Xcode workspace, plus Cerebro/PythonRequirements.txt and README.md.

The project pins MLX-family packages. The inspected documentation records `mlx-swift 0.31.3`, `mlx-swift-lm 3.31.3`, `swift-tokenizers-mlx 0.3.0`, and `swift-hf-api-mlx 0.2.0`. Pinning matters: model-loading APIs, tokenizers, Metal kernels, and model configurations evolve together.

```
1 xcodebuild -project Cerebro.xcodeproj \  
2   -scheme Cerebro -configuration Debug \  
3   -destination 'platform=macOS' \  
4   -derivedDataPath /tmp/CerebroDerivedData \  
5   CODE_SIGNING_ALLOWED=NO build
```

Install the Xcode Metal component if MLX compilation requests it. Configure DepthAI through Cerebro's Python Settings rather than inserting a personal interpreter path. Download local models before an offline show and test cold load, warm load, sustained memory, thermal behavior, camera reconnect, and stop behavior on the production Mac.

Enter through AppDelegate, then find ownership

`AppDelegate` is where process-lifetime services belong: secure control/video listeners, camera helper supervision, optional dependency discovery, and coordinated shutdown. `ROBMainViewController` connects those services to windows and UI state. `ROBSerialBox` owns hardware channels. `CameraViewController` owns presentation and perception work, while `CameraManager` owns camera-provider mechanics.

The essential ownership questions are:

1. Who starts this resource?
2. Who stops it on window closure, disconnect, camera replacement, and app termination?
3. Which queue or actor mutates it?
4. How are late callbacks from a retired generation rejected?
5. What bounded state survives when the consumer is slower than the producer?

A robot app should not rely on `deinit` as its only shutdown protocol. Cameras, file descriptors, subprocess pipes, `Network.framework` connections, timers, model tasks, and `SceneKit` nodes all need explicit lifecycle edges.

Dissect ROBSerialBox: the hardware boundary

SOURCE TRAIL – ANALYZING NOW: Cerebro/ROBSerialBox.h and Cerebro/ROBSerialBox.m, especially `initialize_connection`, `usbSerialPortPaths`, `connectToDetectedBase`, `probeBaseFirmwareAtPath`, `consumeBaseSerialBytes`, `renderController`, and `stopBaseMotionAndDropHeartbeat`.

ROBSerialBox is both valuable and overloaded. Its header exposes serial selection, tread and flipper controls, neck and torso commands, Pololu Maestro and Tic operations, two AMBER-arm command families, controller snapshots, and legacy Head/Torso surfaces. Treat it as an archaeological map of physical integration.

At initialization it sets every descriptor to -1, initializes demanded and actual speeds, creates a bounded Base receive buffer, begins Base discovery asynchronously, identifies the Maestro, starts a 100 ms controller timer, and later starts AMBER log-tail helpers. The retired Head and Torso Arduino UI remains visible, but automatic opening is commented out because only Base is presently installed.

File descriptors are state machines

A descriptor is not merely an integer. It moves through closed, opening, configured, readable/writable, failed, and closing states. -1 is the closed sentinel. Every read and write must use a stable ownership rule; otherwise a reconnect can close a descriptor while an old thread is writing, producing the historical `Bad file descriptor` flood.

The current low-level serial configuration uses POSIX `open`, `termios`, `select`, `read`, and `write`. Cocoa objects may format diagnostics, but blocking I/O must not run on the main thread. A modern refactor should wrap each channel in a serial executor or actor-like Objective-C object and expose typed commands rather than public descriptor-dependent methods.

```
1 if (serialFileDescriptor != -1) {  
2     write(serialFileDescriptor, bytes, length);  
3 }
```

That check alone does not make concurrent close/write atomic. The durable pattern is one queue owning open, read, write, and close, plus an incrementing connection generation attached to callbacks.

Discover the Base Arduino by firmware response

Cerebro enumerates IOKit serial services, accepts only `/dev/cu.usbmodem*` and `/dev/cu.usbserial*`, and sorts candidates. It opens each candidate at 250,000 baud, pulses DTR to reset an Arduino, sends no probe command bytes, and waits up to 15 seconds for the existing startup line:

```
1 BEGIN BASE STARTUP SEQUENCE
```

Only the matching descriptor becomes `serialFileDescriptor_base`. This is better than remembering a `/dev/cu.*` name because hub topology can rename ports. It also avoids sending an accidental motion-like byte to an unidentified device. The startup string is role discovery—not authentication, wiring validation, or proof that motion is safe.

The probe retains at most 8 KiB and trims older bytes. The running line parser caps unterminated input similarly. Those small limits matter: a noisy serial device must not grow application memory forever.

Parse lines before interpreting meaning

`consumeBaseSerialBytes` accumulates arbitrary read chunks, extracts newline-delimited records, and then calls `handleBaseSerialLine`. This respects a fundamental serial fact: one read is not one application message.

The parser recognizes existing warning strings such as `WARNING! FRONT` and `WARNING! BACK`. It also accepts a proposed numeric six-sensor line beginning `ROB:IR=` when available. Each field must contain only decimal digits and be at most 1000. Legacy warnings expire visually; silence means **clearance unknown**, never **path clear**.

Separate devices by identity and protocol

Base discovery uses a firmware banner because that is what the existing sketch emits. Maestro discovery instead inspects USB vendor, product, interface name, and interface number, preferring the named command port. The Tic waist-rotation path invokes validated `ticcmd` arguments. These are three different identity strategies because the devices expose different trustworthy evidence.

Do not create a universal “pick the first USB port” helper. Build a device registry whose match result includes device role, evidence, path, generation, capabilities, and last error. A UI can then report “Base verified by startup banner” and “Maestro matched by USB identity” instead of showing a misleading green dot.

Translate controller intent without extending stale motion

controllerPassthrough turns controller state into the legacy Base frame and actuator targets. renderController runs at 10 Hz while snapshots are expected at 5 Hz. Cerebro expires controller snapshots after three missed updates, writes one neutral/braked frame, and then stops Base USB writes so the Arduino's own heartbeat deadman can expire.

This is a two-layer timeout:

```
1 controller freshness expires
2   ↓
3 Cerebro emits one neutral frame
4   ↓
5 Cerebro stops refreshing Arduino heartbeat
6   ↓
7 Base firmware independently de-energizes motion
```

Never solve packet loss by endlessly replaying the last nonzero command. The more powerful the actuator, the more important command age, authority owner, and stop semantics become.

Control Maestro neck servos and Tic torso rotation

SOURCE TRAIL – ANALYZING NOW: ROBSerialBox.m methods applyVisionNeckPan:tilt:, applyVisionTorsoActive:rotation:, waistRotationSliderAction:, exitSafeStartWaistRotationToggle:, and energizeToggle:.

Vision Pro head orientation arrives as normalized intent. Cerebro clamps it, maps it into Maestro target units, avoids redundant writes, and sends compact protocol bytes. Torso follow maps bounded orientation into Tic position units. The Tic must exit safe start and be energized before accepting motion; the UI mirrors those states and its full rotation range.

Commanded position is not measured position. On boot, an actuator controller can believe a coordinate origin that does not match the physical pose. The book's rule is therefore:

```
1 commanded target ≠ encoder feedback ≠ visually estimated pose
```

Store each with source, timestamp, confidence, and calibration generation. Never overwrite one with another just because they share units.

Bridge Objective-C and Swift deliberately

SOURCE **TRAIL** **–** **ANALYZING** **NOW:** Cerebro/Cerebro-Bridging-Header.h, ROBMainViewController.mm, ROBAl.swift, and the @objc entry points in newer Swift services.

Objective-C callers need stable selector-shaped APIs and Foundation-compatible types. Swift implementation types can keep actors, `Sendable`, async functions, and enums internally, then expose a façade that translates callbacks and notifications.

Use these boundaries:

- cross with `NSString`, `NSData`, `NSNumber`, `NSArray`, `NSDictionary`, and explicitly Objective-C-compatible classes;
- turn Objective-C delegate events into immutable Swift values immediately;
- dispatch UI mutation to the main actor/main queue;
- never expose an MLX tensor, `CVPixelBuffer` lifetime assumption, or actor-isolated mutable object through the bridge;
- make ownership weak where the caller and service retain one another;
- give errors a bounded, user-safe category and keep secrets out of descriptions.

Capture RGB with AVFoundation

SOURCE TRAIL – ANALYZING NOW: Cerebro/CameraManager.swift types CameraSource, CameraSourceState, CameraFrameSet, and its AVFoundation capture methods.

The AVFoundation path discovers a camera, chooses a format, configures AVCaptureSession, and receives CMSampleBuffer frames. Configuration should occur inside beginConfiguration/commitConfiguration; capture callbacks belong on a dedicated queue; UI work belongs on the main actor.

CMSampleBuffer preserves image buffer and timing. Keep it intact as long as possible. Converting every frame into UIImage, then CIImage, then a new pixel buffer creates copies and allocation churn. Consumers should receive one frame set and select the cheapest representation they actually need.

Camera lifecycle is demand-driven. Local preview, Gemini sampling, MLX vision, and Vision Pro streaming may each create demand. The session should run when at least one demand exists, not restart every time a checkbox changes. A run generation prevents callbacks queued by an old session from contaminating a replacement session.

Acquire aligned RGB-D from Luxonis

SOURCE TRAIL – ANALYZING NOW: `docs/depth-camera.md`, `Cerebro/Webcam_color.py`, `CameraManager.swift`, `ROBPythonRuntime.m`, and `Tests/DepthCameraIPCFixtureTests.py`.

The primary OAK path uses DepthAI 3.8.0 in a supervised Python helper. The helper alone owns the device, builds Camera + Depth + Sync nodes, requests 640 by 400 undistorted RGB, aligns depth to RGB, and sends paired frames through a private Unix-domain socket.

The versioned CDP1 packet contains bounded metadata, RGB888 bytes, and little-endian unsigned 16-bit depth in millimeters. Zero depth is invalid. Cerebro validates magic, version, dimensions, lengths, pixel formats, and maximum allocation before creating CoreVideo objects.

The out-of-process boundary is a reliability feature. USB disconnects, SDK exceptions, malformed packets, and native-library faults disable/restart the camera provider without taking robot control down. Retries use bounded backoff. AVFoundation remains an RGB-only fallback, while legacy OAK UVC fallback is explicit opt-in so two providers never fight for the same device.

The builder identifies two present Luxonis roles: an OAK-D Pro Wide in the body and an autofocus OAK-D Pro at the head. Do not infer the exact sensor option from the enclosure. At discovery, save `productName`, `boardName`, board revision, camera sensor names, USB speed, calibration hash, focus capability, and firmware/API version. The RVC2 OAK-D Pro W family uses global-shutter monochrome stereo sensors and offers a wide color-camera option; its active-stereo projector and flood illumination add power and thermal load. A camera that enumerates successfully can still be on a USB 2 link, underpowered, thermally constrained, out of calibration, or owned by the wrong provider.

Separate host inference from OAK inference

ROB has three distinct model execution paths:

1. **DepthAI device graph:** camera, stereo depth, synchronization, and an optional compiled Myriad X `.blob` execute in the OAK pipeline. Only bounded results and RGB-D frames cross the USB process boundary.
2. **Apple Vision/Core ML:** `ROBDynamicDetectorRegistry` runs built-in Vision requests or registered Core ML models on the Mac. It compiles an `.mlmodel` to `.mlmodelc` when needed, creates `MLModel`, wraps it in `VNCoreMLModel`, and executes `VNCoreMLRequest` at a configured admission rate.
3. **MLX language/vision models:** actor-owned local LLM, VLM, and embedding containers run on Apple silicon for delayed reasoning and stage context, never the motor loop.

The same word *model* appears in all three paths, but their files, devices, input tensors, output schemas, latency, memory, and failure scopes differ. Record those facts in every model manifest.

Understand alignment

Depth alignment means pixel (x, y) in the depth plane refers to the same camera ray as pixel (x, y) in RGB after calibration and resampling. It does not mean every depth is valid. Reflective, transparent, too-near, too-far, or texture-poor surfaces can return zero or noisy values.

`CameraDepthFrame.distanceMillimeters(x:y:)` validates coordinates and byte offsets. Downstream code rejects implausible distances. A person's estimated range is the median of a small interior grid, not a single fragile pixel.

Build a bounded perception pipeline

SOURCE **TRAIL** – **ANALYZING** **NOW:** `CameraViewController.swift,`
`HumanBodyPose3DDetector.swift,` `HumanBodySkeletonRenderer.swift,` and `ROBSceneSnapshot.swift`.

Camera rate is not inference rate. A 20–30 fps camera may feed a body request only when the previous request has finished. The UI may display the newest RGB frame while depth visualization, pose inference, MLX VLM sampling, Gemini JPEG encoding, and remote H.264 encoding operate at different bounded rates.

The reliable pattern is a latest-value mailbox:

```
1 producer offers frame N
2 worker busy? replace pending frame, do not append
3 worker finishes
4 consume newest pending frame, if any
```

This bounds latency and memory. A robot benefits more from the newest frame than from processing a five-second-old queue perfectly.

Vision observations are normalized and carry confidence. SceneKit nodes should be reused or replaced as a group, not appended forever. Heavy geometry creation stays off the main thread where APIs allow; final scene mutation is serialized. Wrap temporary per-frame Objective-C objects in autorelease pools on long-running queues.

Render perception without leaking memory

`CameraViewController` combines preview, pose overlays, depth coloring, point-cloud geometry, camera-pose visualization, and notifications. Every render path needs an explicit budget:

Resource	Bounded strategy
Raw RGB	newest frame only for slow consumers
Depth	fixed dimensions and validated byte count
Vision requests	one in flight per detector
SceneKit nodes	reuse named roots; replace children atomically
Point cloud	stride/downsample based on detail setting
CI rendering	reuse <code>CITexture</code> ; avoid per-frame context creation
Pixel buffers	pool where encoding requires conversion
ML models	one actor-owned container per selected model
Semantic memory	maximum 200 in-process entries
Logs	rate-limit repeated camera/model/device failures

Use Instruments Allocations and Memory Graph while toggling camera sources repeatedly. A stable single-frame memory profile is insufficient; test 30 minutes of capture, disconnect/reconnect, window close/reopen, model load, and remote video subscription.

Turn observations into SceneSnapshot

SceneSnapshot is the typed boundary between real-time sensing and language reasoning. It contains tracked people, objects, gestures, free-space regions, arm joint poses, reverse camera pose, camera quality, and aggregate confidence. Missing data stays missing.

The store locks only long enough to replace small value arrays or take a snapshot. It calculates three depth free-space bands—left, forward, right—using strided samples, valid-depth confidence, an 800 mm clearance test, and a 75 percent clear-fraction threshold. Fresh lidar regions are merged only for 1.5 seconds after their update.

Its language-model serialization wraps sorted JSON as **untrusted sensor data, not instructions**. That distinction prevents text observed in the scene from silently becoming a model instruction. A macOS 26 Foundation Models interpreter can return a typed AssistantIntent, but the intent contains high-level suggestions rather than tread or joint commands.

Run MLX privately on Apple silicon

SOURCE **TRAIL** – **ANALYZING** **NOW:** Cerebro/ROBMLXRuntime.swift, ROBMLXImprovisationProvider.swift, ROBMLXStageObservation.swift, docs/mlx-local-ai.md, and Package.resolved.

ROBMLXEngine is an actor. This serializes model-container state, loading, diagnostics, VLM admission, embeddings, and semantic memory without blocking the main actor. The initial local LLM is `mlx-community/Llama-3.2-1B-Instruct-4bit`; vision uses `mlx-community/Qwen2-VL-2B-Instruct-4bit`; retrieval uses TaylorAI/`gte-tiny`.

`ensureLLMReady` avoids duplicate loading. `generate` creates user input, applies token and temperature limits, records latency and throughput, and keeps model output outside motor control. Model downloads occur on first use through the Hugging Face cache, so show-day preparation must prefetch and prewarm.

MLX VLM—sometimes abbreviated incorrectly as VLX

The repository integrates an MLX vision-language model through `VLMModelFactory`; “VLX” in conversation should be read as this MLX VLM path, not as a separate framework. `offerVisionFrame` is nonblocking and admits at most one selected image every five seconds, never faster than three seconds. The VLM returns observational stage context. It cannot actuate hardware.

Sampling protects three budgets at once: GPU/Metal memory, thermal headroom, and perception latency. Record active and peak MLX Metal memory, generated tokens per second, model load state, sampled-frame count, and last error. Clear temporary caches only at safe lifecycle boundaries; do not unload/reload a model between cues.

Read the MLX execution path line by line

The important code is small enough to narrate without treating the framework as magic:

```
1 public actor ROBMLXEngine { ... }
2 GPU.set(cacheLimit: 128 * 1024 * 1024)
3 GPU.set(memoryLimit: 6 * 1024 * 1024 * 1024)
4 let container = try await loadLLM(modelID: modelID)
5 let input = try await container.prepare(input: UserInput(prompt: prompt))
6 let stream = try await container.generate(input: input, parameters: ...)
7 for await event in stream { ... }
```

1. actor makes the engine the single owner of mutable model state. It does not make every referenced framework type automatically thread-safe.
2. Cache and memory limits bound MLX's use of unified memory so vision, UI, encoding, and control retain head-room. They are caps, not reservations or proof against system memory pressure.
3. `loadLLM` returns the already loaded container when the model ID matches, otherwise it records loading/download diagnostics and builds a container through `LLMModelFactory`.
4. `prepare` tokenizes the prompt and constructs the model input. Prompt text is data; length, provenance, and privacy must be controlled before this line.
5. `GenerateParameters` bounds output tokens and temperature. Low temperature reduces randomness but does not create truth.
6. `generate` returns an asynchronous event stream. `.chunk` appends text, `.info` records throughput, and `.toolCall` is rejected in the local improvisation path.
7. `Task.checkCancellation()` prevents an abandoned stage cue from consuming unlimited time, while request correlation prevents its late text from controlling a replacement cue.
8. The returned string is still untrusted. A separate strict codec must decode an allow-listed schema before a behavior layer can even consider it.

The VLM path adds an image but keeps the same boundary. `offerVisionFrame` checks a global enable, one-in-flight flag per source, and a minimum interval. It returns immediately; the actor later prepares `UserInput(prompt: images:)`. The prompt demands one minified JSON object, the codec extracts only that object, validates types and bounds, strips person identity, attaches confidence and time, and publishes delayed context. `currentStageContext` refuses old or low-confidence observations and explicitly labels accepted facts as uncertain, non-executable context.

Teach ROB to recognize a new object without teaching it to trust itself

SOURCE TRAIL – ANALYZING NOW: `ROBDataSetManager` in `ROBSceneSnapshot.swift`, `TrainProjectModel.py`, the model-manifest loader and `YoloSpatialDetectionNetwork` branch in `Webcam_color.py`, and `ROBDynamicDetectorRegistry.swift`.

The repository already sketches a learning-and-compilation chain:

```

1 human chooses project and class
2   -> bounded JPEG + normalized YOLO label
3   -> dataset folders + classes.txt + data.yaml
4   -> YOLOv8n fine-tuning on Apple MPS
5   -> best.pt
6   -> ONNX export, opset 12, input 640x400
7   -> blobconverter, six SHAVES
8   -> yolov8_<project>_6shave.blob + JSON manifest
9   -> DepthAI helper restart
10  -> YoloSpatialDetectionNetwork on OAK-D Pro

```

The dataset manager first validates a project name, normalizes the class label, rejects non-finite or out-of-bounds boxes, JPEG-encodes the selected image, writes image and YOLO label atomically, and updates class metadata. A label line is `class_index center_x center_y width height`, all normalized to the unit square. This is annotation, not learning yet.

`TrainProjectModel.py` then performs these decisions:

- **safe_project_name:** prevents path syntax in a project identifier.
- **require_within:** rejects dataset paths that escape the selected root after resolution.
- **validated_class_names:** requires contiguous unique class IDs and exact nc agreement.
- **validate_dataset:** requires distinct train/validation image directories, labels, and matching `classes.txt`.
- **YOLO("yolov8n.pt"):** starts from pretrained weights; record their exact hash and license.
- **model.train(... epochs=50, imgsz=640, device="mps"):** fine-tunes on the Mac; fix random seeds and save the environment and hyperparameters for reproducibility.
- **Export best.pt to ONNX:** creates an interchange graph; validate its outputs against the PyTorch model.
- **blobconverter.from_onnx(... shaves=6):** compiles for Myriad X; compilation success is not accuracy evidence.
- **Write the JSON manifest:** binds labels, parser type, input geometry, class count, and thresholds to the blob.
- **Replace the final blob and manifest:** currently makes the artifact discoverable; production needs a signed staged release and rollback.

There is an honest integration gap: the dataset manager currently writes both `train` and `val` paths to `./images/train`, while the trainer correctly rejects identical training and validation directories. This prevents the automatic chain from being a trustworthy one-button learner. Fix the data lifecycle first: capture separate train, validation, and locked test sets by collection session or scene, so nearly identical video frames cannot leak across splits.

The DepthAI helper resolves `yolo_v8_<project>_6shave.blob`, loads the adjacent manifest, checks that the manifest stem matches the filename, and configures a `YoloSpatialDetectionNetwork`. Camera settings can request a restart to hot-swap the model. That restart is a deployment boundary and should happen only while the affected perception feature is unavailable and motion policy no longer depends on its results.

`ROBDynamicDetectorRegistry.registerCoreMLModel` is a separate Mac-side path. If the URL is not already `.mlmodelc`, `MLModel.compileModel` compiles it; `MLModel(contentsOf:)` loads it; `VNCoreMLModel(for:)` adapts it to Vision; a lock publishes it to future requests. The current method does not yet enforce signer, manifest, input geometry, output schema, class allowlist, benchmark, duplicate identity, or rollback. Treat it as a development hook until those checks exist.

A safe future learning architecture

ROB should never train on an observation and immediately let the new model steer or move an arm. Use a promotion pipeline:

1. **Consent and purpose:** collect only authorized images for one named task; retain provenance and deletion controls.
2. **Annotation review:** a human checks boxes/classes and removes sensitive or ambiguous samples.
3. **Split by event:** keep train, validation, and locked test scenes disjoint; add hard negatives and the surfaces/lighting ROB will meet.
4. **Reproducible training:** pin code, base weights, dependencies, seed, hardware, hyperparameters, and dataset hashes.
5. **Cross-runtime comparison:** compare PyTorch, ONNX, compiled OAK blob, and/or Core ML outputs on the same golden images.
6. **Acceptance gates:** require per-class precision/recall, confusion matrix, calibration, latency, memory, thermal, and adversarial/edge-case results against declared thresholds.
7. **Signed candidate:** package model, manifest, metrics, training record, limitations, and signer; never overwrite the last known-good artifact in place.
8. **Shadow mode:** run the candidate beside the approved model without granting control authority; log disagreements with bounded, privacy-reviewed samples.
9. **Human promotion:** a named reviewer promotes one version during a disarmed maintenance state.
10. **Rollback and expiry:** health monitoring can disable the candidate immediately; the operator can restore the prior signed release; model validity expires when camera calibration, geometry, or mission changes.

The dynamic intelligence is the system that can create, test, explain, select, and revoke a model. The model itself remains one fallible component.

Add bounded semantic memory

`remember` embeds short text. `retrieve` embeds a query, calculates cosine similarity, sorts matches, and returns a limited result set. The current store holds at most 200 entries and is process-local.

This is retrieval, not truth. Each memory needs provenance and a future persistent design would require encryption, deletion, retention, model-version tracking, and re-embedding policy. Never store raw camera images or secrets merely because embeddings feel abstract.

Constrain local model output

ROBMLXImprovisationProvider asks for exactly one JSON document with a fixed schema, allow-listed beat, allow-listed delivery, and bounded offline line. It then decodes and validates the output independently. Prompt instructions are a generation aid; the codec is the trust boundary.

```
1 {  
2   "schema": "com.orbitusrobotics.local-improvisation-plan",  
3   "version": 1,  
4   "beat": "robot_joke",  
5   "delivery": "deadpan",  
6   "offline_line": "My timing is local; your laughter is distributed."  
7 }
```

Cancellation is correlated by request ID. A completion that arrives after cancellation is discarded. Health checks load the selected model without granting it a physical tool. The alternate llama.cpp provider uses the same domain protocol and loopback HTTP, preserving the stage coordinator and validation logic.

Integrate Gemini Live without coupling safety

SOURCE TRAIL – ANALYZING NOW: Cerebro/ROBAI.swift, GeminiRoboticsProtocol.swift, ROBGeminiDiagnosticsWindowController.swift, docs/gemini-robotics-live.md, and Tests/GeminiRoboticsProtocolFixtureTests.swift.

ROBAI is the Objective-C-facing façade over a Swift live-session actor. Configuration comes from explicit environment settings and credentials. Connection, microphone streaming, camera streaming, and the optional robot-action tool are separate effective switches.

The audio adapter converts captured samples to the PCM format required by the live protocol and preserves event ordering. The video adapter samples frames and produces bounded JPEG data. Neither should enqueue unlimited media. Diagnostics report effective input mode, counters, transcription state, model turns, and redacted errors without logging credentials or private payloads.

The secure WebSocket setup, realtime messages, transcription fragments, tool calls, interruptions, deadlines, and turn completion are parsed into typed events. A reconnect starts a new session generation; late callbacks from the old socket cannot complete the new turn.

Use GEMINI_EPHEMERAL_TOKEN where an issuing service exists; a development GEMINI_API_KEY is supported. Never place either in source, a show JSON file, a screenshot, or a book. Camera and microphone remain off unless the operator explicitly enables them.

Treat model tools as proposals

The optional `robot_action` path validates a versioned proposal, routes it to the operator approval surface, correlates result messages, and returns only terminal outcomes to Gemini. Approval currently records operator intent; it does not prove physical execution exists.

Stage-originated Gemini turns reject non-stop physical tool calls even after the cue has timed out. `stop_motion` uses a priority path: stop coordinators, write one neutral/braked Base frame, drop the heartbeat, and return control authority to Brain. AMBER arm hold remains explicitly unverified.

A safe model architecture is:

```
1 model text/tool suggestion
2   ↓ strict decoding
3 typed proposal
4   ↓ policy and operator approval
5 bounded action state machine
6   ↓ hardware-specific executor
7 measured result or explicit unavailable
```

Never connect model tokens directly to `write`, `ticcmd`, a servo target, shell command, or AMBER packet.

Stream camera video to Vision Pro separately

SOURCE TRAIL – ANALYZING NOW: `ROBVideoServer.swift`, `ROBVideoProtocol.swift`, `ROBCameraH264Encoder.swift`, `docs/vision-pro-video.md`, and Volume 7.

Cerebro advertises `_robvideo._udp` with ALPN `robvideo/1`, separately from `_robctl._udp` control. Both use TLS 1.3 and the paired certificate/secret relationship, but video uses its own authentication transcript and requires a matching live operator control session.

The initial profile is H.264 AVCC, at most 960 by 540, 20 fps, and 1.5 Mbps, with B-frames disabled and key frames at least every second. One raw frame may wait; one encoder output may wait; one send may be in flight. Congestion drops work instead of accumulating latency. Receiver recovery can request a key frame and codec configuration.

Volume 7 explains the other half: Vision Pro discovery, pinned identity, subscription, defensive framing, H.264 sample-buffer creation, `AVSampleBufferDisplayLayer`, controller input, head pose, speech-to-text, and dead-man authority. Read the two books side by side whenever changing the wire contract.

H.264 without hand-waving: pictures, access units, and NAL units

H.264 is a video coding format; it is not by itself a network protocol. The encoder turns images into coded syntax. The Network Abstraction Layer divides that syntax into **NAL units** suitable for storage or transport. One displayed picture is commonly represented by an **access unit**, which can contain several NAL units.

For H.264/AVC, the first byte of each NAL unit is:

1	bit 7	bits 6..5	bits 4..0
2	forbidden_0	nal_ref_idc	nal_unit_type

The current validator requires the forbidden bit to be zero and masks the low five bits to find the type. Useful types in ROB's profile include:

- **Type 1 – non-IDR coded slice:** predictive picture data that may depend on earlier reference pictures.
- **Type 5 – IDR coded slice:** a random-access recovery picture; the current key-frame flag must agree with the presence of type 5.
- **Type 6 – SEI:** supplemental information; not treated as the required video-coding payload.
- **Type 7 – SPS:** the sequence parameter set, carrying coded geometry, profile/level, and sequence-wide decoding facts.
- **Type 8 – PPS:** the picture parameter set, carrying picture-level decoding configuration referenced by slices.
- **Type 9 – access-unit delimiter:** an optional delimiter; ROB does not depend on it for framing.

SPS and PPS are configuration, not ordinary image pixels. Cerebro extracts them from `CMFormatDescription` on a key frame and sends them in a separate codec-configuration message. The receiver uses at least one SPS and one PPS to create `CMVideoFormatDescription`, then validates coded and presentation dimensions against the negotiated stream before allocating ongoing decode state.

Annex B and AVCC are two wrappers around NAL units

An Annex B byte stream finds NAL units with `00 00 01` or `00 00 00 01` start codes. An AVCC sample prefixes each NAL unit with a 1-, 2-, or 4-byte big-endian length. VideoToolbox gives ROB length-prefixed AVCC samples, and the exact prefix width comes from the format description.

1	AVCC access unit with 4-byte lengths		
2			
3	00 00 02 8A	[650-byte NAL unit]	
4	00 00 00 31	[49-byte NAL unit]	

The prefix is not part of the NAL unit. The first byte after each prefix is the NAL header. `validateLengthPrefixedAccessUnit` walks exactly this structure: read the configured prefix width, build a big-endian length, reject zero/truncation/overflow, inspect the NAL type, advance by exactly that length, and require at least one type 1-5 unit. A key-frame sample must contain type 5; a non-key sample must not claim it.

Do not scan an AVCC payload for Annex B start-code patterns. Compressed payload bytes can coincidentally contain them, and the representation already has authoritative lengths. Convert formats only at one named, tested boundary.

Read the encoder from input pixel to encoded bytes

`ROBCameraH264Encoder` performs this sequence:

1. Validate width, height, total pixels, frame rate, and bitrate against hard caps.
2. Create a `VideoToolbox H.264` compression session whose destination pixels are `NV12` video-range (`420YpCbCr8BiPlanarVideoRange`).
3. Request real-time operation, disable frame reordering, prefer `Constrained Baseline`, set expected frame rate/average bitrate, and request a key frame at least once per negotiated second.
4. Create one reusable pixel-transfer session in letterbox mode.
5. Admit samples at the requested rate using monotonic uptime. Do not encode every callback simply because the camera produced it.
6. Allocate from the compression session's pixel-buffer pool with a threshold of six. When the pool is exhausted, drop the frame and request a future recovery key frame.
7. Scale/letterbox the source into the destination buffer and call `VTCompressionSessionEncodeFrame` with a monotonic presentation time.
8. In the callback, claim completion exactly once because `VideoToolbox` can report a synchronous drop through more than one path.
9. Copy the bounded `CMBlockBuffer` bytes into owned `Data`; callback-owned memory must not escape by reference.
10. Read the not-sync attachment to determine whether `VideoToolbox` marked the sample as a key frame.
11. Extract the NAL length width from `CMFormatDescription`, validate every AVCC NAL, and on a key frame copy the parameter sets.
12. Emit sequence, wall-clock capture time, presentation time, duration, key-frame flag, optional parameter sets, prefix width, and payload.

Disabling B-frame reordering means decode order follows presentation order for this low-latency profile. It costs some compression efficiency but removes a reorder queue and simplifies recovery. `Constrained Baseline` is requested with documented fallback for unsupported properties, so runtime diagnostics should record the format actually returned.

ROB's transport is custom reliable QUIC, not RTP

RFC 6184 describes carrying H.264 NAL units over RTP using single-NAL packets, aggregation packets such as STAP-A, and fragmentation units such as FU-A. ROB's current path does none of those. It places complete AVCC access units inside the inner RBVD binary message, then places that message inside an outer ordered RVID frame on a reliable QUIC stream.

```

1 QUIC/TLS byte stream
2   RVID 32-byte connection frame
3     RBVD 92-byte media header
4       AVCC access unit
5         [length] [NAL] [length] [NAL]...
```

The outer RVID layer identifies message kind and bounds the next payload before allocation. The inner RBVD layer binds media to a session UUID and subscription UUID and carries codec, sequence, capture time, presentation time,

duration, timescale, configuration generation, NAL-length width, key-frame flag, and payload length. The current caps are 64 KiB for configuration and 2 MiB for one access unit.

Reliable ordering preserves every byte, but a lost network packet can delay later video through head-of-line blocking. ROB limits the damage with a separate video connection, newest-only admission before encode, one queued output, one send in flight, deadlines, and stream teardown when a peer stops reading. Control uses a different QUIC connection, so a delayed video access unit does not sit in front of a stop message, although both still compete for Wi-Fi airtime.

Why a missing access unit triggers IDR recovery

Non-IDR slices may reference decoded pictures that the receiver no longer has. After a sequence gap, decoder error, renderer flush, or configuration-generation change, displaying more dependent frames can create corruption. The receiver enters `needsKeyFrame`, drops predictive units, rate-limits feedback, and waits for new configuration plus a valid type-5 IDR access unit. The server's encoder sets a thread-safe `force-next-key-frame` flag. Recovery is a state machine, not merely "send another picture."

Test NAL handling with generated fixtures: zero length, truncated prefix, oversized NAL, forbidden bit set, no VCL type, key flag without type 5, type 5 without key flag, SPS without PPS, dimension-changing SPS, configuration change without flush, sequence gap, and repeated key-frame requests. Run them without a camera or network before testing live media.

Excavate the Kinect and OpenNI past

SOURCE TRAIL – ANALYZING NOW: the `ROBNIManager` and `TaskControllers` directories: `ROBNIManager.mm`, `FreenectPCL.mm`, `k2g.h`, `viewer.*`, `shaders`, duplicated `* 2.*` artifacts, and initial commit `4c4f1d4` from 2025-08-05.

The repository contains an older depth-vision stack built around Kinect/OpenNI/NiTE-style naming, `libfreenect/PCL` bridges, C++ point clouds, GL viewers, skeletal tracking, shaders, and serialized task-controller processes. These files arrived in the current Git history with the initial import; that history does not establish when the original experiments were written.

The artifacts teach important lessons:

- camera SDKs age faster than domain concepts;
- C++ exception and ABI boundaries can destabilize an app process;
- duplicated filenames with `2` indicate migration debt, not alternate APIs to call casually;
- shader and viewer code may embody useful coordinate conventions even when its device path is retired;
- binary Core ML models such as `ObjectDetector.mlmodel` need provenance, input/output documentation, license review, and a reproducible evaluation set before reuse.

Do not delete ancient code merely because it looks strange. First map whether project files still compile it, whether runtime selectors reference it, whether a modern test covers its surviving behavior, and whether historical knowledge should move into documentation. Then retire it in a small commit with a rollback point.

Compare Kinect-era and Luxonis-era design

The old path intertwined device SDK, C++ processing, visualization, and task launching. The new Luxonis path defines a provider contract and isolates the SDK in a supervised helper. The improvement is not simply “new camera has better depth.” It is failure containment and replaceability.

Question	Historical depth stack	Current OAK design
Device boundary	native C++/framework integration	supervised Python process
Transfer	internal objects/task artifacts	versioned local CDP1 bytes
RGB-depth pairing	implementation-specific	timestamp-sync and RGB alignment
Crash scope	potentially Cerebro process	helper can restart independently
Fallback	separate historical controllers	explicit AVFoundation RGB-only path
Contract tests	sparse artifacts	malformed/oversized IPC fixtures

The next evolution could replace Python with an XPC/native helper while preserving CameraFrameSet. Stable contracts let implementation technology change without rewriting perception and UI.

Build a reliable light-saber battle trainer

SOURCE **TRAIL** – **ANALYZING** **NOW:** `ROBSaberChoreography.swift`,
`StageShows/ProgressiveSaberTraining.robshow.json`, `GalacticSaberBattle.robshow.json`,
`ROBAmblerB1Kinematics.swift`, and `Tests/ROBSaberChoreographyFixtureTests.swift`.

The trainer uses named gestures and a progressive profile rather than embedding arbitrary joints in show files. `ROBSaberChoreography` maps an allow-listed gesture name to bounded Cartesian transforms and reports a requested duration. The real AMBER kinematics and calibration layers must still validate reachability and transform frames before execution.

A good training progression begins with guard pose and slow single-plane movements, then alternates direction, timing, and combinations. It should enforce a participant boundary, soft prop policy, low speed, reduced force, guarded robot workspace, independent emergency stop, and a dry-run visualization. “Battle” is theatrical choreography, never permission for autonomous contact.

The perception loop can score timing and approximate pose without commanding the arm from pixels. It observes participant pose, estimates phase, compares against the authored target, and chooses spoken coaching. Motion remains authored and bounded.

Engineer the comedy show as a state machine

SOURCE TRAIL – ANALYZING NOW: `ROBStageShowProtocol.swift`, the coordinator and window controller, the `OrbitusTenMinuteComedy` file under `StageShows`, and `ROBStageShowFixtureTests.swift` under `Tests`.

A `.robshow.json` document contains typed cues such as speech, wait, checkpoint, gesture, and Gemini turn. The codec rejects unknown or dangerous content. Show files cannot contain raw joint values, hosts, ports, or shell commands.

The coordinator advances one cue at a time and records what it is awaiting: speech completion, checkpoint, gesture, local model, Gemini, or timer. Each asynchronous request has an ID and deadline. Cancellation clears scheduled work and rejects late completion.

Four execution modes preserve rehearsability:

Mode	Local model	Gemini	Spoken fallback
Dry Run	no	no	none; validate without side effects
Offline	no	no	authored line
Local	optional MLX/llama.cpp	no	validated local line, then authored line
Adaptive	optional	yes	Gemini, validated local line, authored line

The comedy file targets roughly ten minutes and includes multiple adaptive moments. A clever line is never allowed to block the show forever: timeouts, authored fallback, and explicit cue advancement are part of the creative system.

Test at contracts, not only windows

The repository includes fixture executables for Gemini messages, robot-action proposals, secure control, video framing and encoding, DepthAI IPC, local improvisation, stage shows, saber choreography, MLX stage observations, and catalog validity. These tests are valuable because most can run without moving ROB.

For every boundary, test:

1. smallest valid message;
2. largest valid message;
3. truncated header and payload;
4. oversized length before allocation;
5. unknown version, enum, or field;
6. stale sequence/generation/request ID;
7. cancellation before and after callback scheduling;
8. unavailable hardware/model/network;
9. repeated reconnect and shutdown;
10. redaction of secrets and private media.

Then perform a staged hardware test: no-power simulation, powered actuators mechanically isolated, wheels/treads raised, low limits, one subsystem, verified emergency stop, and only then integrated operation.

Profile memory, latency, and thermals

Measure end-to-end age, not just inference duration. Attach capture timestamp to every frame and preserve it through RGB-D pairing, Vision, snapshot, model sampling, encoding, network transfer, and presentation.

Use Instruments Time Profiler, Allocations, Leaks, Metal System Trace, and Points of Interest. Add signposts around camera receive, RGB conversion, depth render, Vision request, VLM admission, model generation, H.264 encode, and main-thread scene update.

Watch for these characteristic failures:

- steadily rising SceneKit nodes or textures;
- a new `CITexture`, formatter, or pixel-buffer pool per frame;
- retained `CMSampleBuffer` objects in asynchronous closures;
- autoreleased Objective-C objects accumulating on worker threads;
- model containers loaded more than once;
- an unbounded array of diagnostics, semantic memories, or network frames;
- main-thread image conversion causing controller lag;
- repeated identical errors overwhelming Xcode and hiding state transitions.

Define production budgets for memory, frame age, UI frame time, model latency, camera restart time, and stop latency. A green light without those measurements is only a connection indication.

Refactor without erasing history

The highest-value seam is to split `ROBSerialBox` by device and responsibility:

```
1 ROBSerialBox façade for existing outlets/selectors|—  
2 ROBBaseSerialChannel|—  
3 ROBMaestroChannel|—  
4 ROBTicWaistController|—  
5 ROBAmberArmGatewayClient|—  
6 ROBControllerCommandArbiter
```

Move parsing and mapping into pure functions with fixtures first. Preserve the Objective-C façade so Interface Builder and existing callers keep working. Introduce typed state snapshots. Move every descriptor to one owning queue. Rate-limit diagnostics. Remove Head/Torso UI only after confirming no nib binding or operator workflow still requires it.

For camera work, preserve `CameraManagerProtocol` and `CameraFrameSet`. For model work, preserve `ROBLocalImprovisationProviding`, `SceneSnapshot`, and strict codecs. For Vision Pro work, preserve the versioned control/video contracts and update both repositories in lockstep.

File-by-file reading route

Read in this order and make notes beside the code:

1. `README.md` and `docs/` for declared behavior and operational boundaries.
2. `AppDelegate.m` and `ROBMainViewController.mm` for process composition.
3. `ROBSerialBox.h/.m` for hardware history, current Base discovery, Maestro/Tic, and arm seams.
4. `CameraManager.swift` and `Webcam_color.py` for provider ownership and RGB-D framing.
5. `CameraViewController.swift`, pose detector, and skeleton renderer for perception/display cost.
6. `ROBSceneSnapshot.swift` for typed world state and confidence.
7. `ROBMLXRuntime.swift`, MLX provider, and local-improvisation protocol for private inference.
8. `GeminiRoboticsProtocol.swift` before `ROBAI.swift`; learn the wire types before the session actor.
9. `ROBStageShowProtocol.swift` before its coordinator; learn allowed data before lifecycle.
10. `ROBSaberChoreography.swift`, AMBER kinematics, and visual calibration together.
11. `ROBVideoProtocol.swift`, encoder, and server, followed by Volume 7's receiver chapters.
12. Every matching fixture test before making a change.

Production review checklist

- ☐ Every serial descriptor has one owner and bounded reconnect behavior.
- ☐ Base identity is detected by its existing firmware response without probe commands.
- ☐ Stale controller input produces one neutral frame and then drops heartbeat.
- ☐ Commanded, measured, and visually estimated poses remain distinct.
- ☐ Camera ownership cannot race between DepthAI and UVC providers.
- ☐ Every IPC and network length is validated before allocation.
- ☐ Slow perception, inference, encoding, and network consumers retain newest-only bounded work.
- ☐ Vision and model work never blocks controller or main-thread responsiveness.
- ☐ SceneKit nodes, contexts, pixel buffers, and model containers remain bounded over long runs.
- ☐ Local and cloud model output crosses strict typed codecs.
- ☐ No language model has direct motor, shell, serial, or arm authority.
- ☐ Stage cues have deadlines, cancellation correlation, and authored fallback.
- ☐ Camera and microphone streaming show explicit effective operator state.
- ☐ Vision Pro control and video protocol versions match Volume 7's client.
- ☐ Historical Kinect artifacts are documented before removal or reuse.
- ☐ Tests pass before any powered-hardware commissioning.

Closing principle

Cerebro is strongest when every subsystem admits uncertainty and owns failure locally. Serial discovery should identify rather than guess. Depth should mark invalid pixels. Vision should preserve confidence. Models should propose typed meaning rather than emit actuator bytes. Stage shows should continue through bounded fallback. Video should drop frames rather than delay control. The Vision Pro should express fresh human intent, while Cerebro and the robot enforce authority, time, and safe limits.

That is how a mixed Swift and Objective-C codebase becomes a dependable robot mind: not through one magical model, but through explicit contracts between imperfect parts.